

Resumo de cada sistema desenvolvido.

Arquivo eu estou comprometido a esclarecer com minhas palavras cada sistema, minhas dificuldades com cada e os objetivos.

TAREFA 1 -

O sistema entra em uma pasta(tarefa)

executa o script .py

Busca usuarios na api.

Filtra as cidades configuradas.

e exibe o total encontrado.;

OBS: a API é estrangeira e não coleta dados de pessoas nessas localidades, implementei cidades brasileiras para uma melhor "exemplificação", mas visto que o sistema .py filtra.

Sistema feito utilizando a api e algumas bibliotecas nativas de PY.

Eu criei uma classe chamada UserCollector para separar as responsabilidades do coletor. Primeiro ela busca os usuários na API pública, depois filtra pelos nomes de cidade configurados, gera estatísticas e salva tudo em JSON com timestamp.

```
PS C:\WINDOWS\system32> cd "C:\programacao\tarefa-01"
PS C:\programacao\tarefa-01> python collector.py
Arquivo gerado: users_20260812_043714.json
Total de usuários encontrados: 0
Cidades pesquisadas: Carapicuíba, São Paulo, Osasco, Barueri
PS C:\programacao\tarefa-01> |
```

TAREFA 2-

Sistema feito para identificar problemas em hardwares, sistema muito exemplificado, funciona como:

CPU maior que 80% = problema

Memória maior que 90% = problema

O sistema foi produzido para ler o input.json, da para facilmente mudar o exemplo se alterar os valores

```
PS C:\Users\victo> cd "C:\programacao\tarefa-02"
PS C:\programacao\tarefa-02> python processor.py
Processamento concluido.
Total de servidores: 3
Servidores com problemas: 1
Arquivo JSON: output.json
Arquivo CSV: output.csv
PS C:\programacao\tarefa-02> |
```

TAREFA 3 -

Esse sistema um pouco mais complexo, trata-se de um monitor de urls capaz de ler uma lista de sites/apis que estão no arquivo config.yaml

Obs: Ao tentar compilar no Vscode na versão do Python 6.0.2 deu erro, foi necessário atualizar para a 6.0.3(não sei informar a origem do erro, mas mudando a versão obtive o resultado esperado).

```

Total de URLs: 3
Sucesso: 2
Falhas: 1
PS C:\programacao\tarefa-03>
PS C:\programacao\tarefa-03> |

```

tarefa 4 -

Na tarefa 4 eu peguei o monitor de URLs da tarefa 3 e tentei rodar dentro de um container docker, mas acabei enfrentando problemas ao rodar na maquina local. Consegui rodar através da alternativa:

```
cd "C:\programacao\tarefa-04"
```

```
python monitor.py
```

```

PS C:\Users\victo> cd "C:\programacao\tarefa-04"
PS C:\programacao\tarefa-04> python monitor.py
{"timestamp": "2026-08-13T00:18:36.444117+00:00", "level": "INFO", "logger": "URLMonitor", "message": "checking_url", "url": "https://jsonplaceholder.typicode.com", "attempt": 1}
{"timestamp": "2026-08-13T00:18:36.515951+00:00", "level": "INFO", "logger": "URLMonitor", "message": "url_checked", "url": "https://jsonplaceholder.typicode.com", "status": "ok", "status_code": 200, "response_time_ms": 71.83, "attempts": 1, "checked_at": "2026-08-13T00:18:36.515909+00:00", "error_type": null, "error_message": null}
{"timestamp": "2026-08-13T00:18:36.516192+00:00", "level": "INFO", "logger": "URLMonitor", "message": "checking_url", "url": "https://api.github.com", "attempt": 1}
{"timestamp": "2026-08-13T00:18:36.765359+00:00", "level": "INFO", "logger": "URLMonitor", "message": "url_checked", "url": "https://api.github.com", "status": "ok", "status_code": 200, "response_time_ms": 249.1, "attempts": 1, "checked_at": "2026-08-13T00:18:36.765290+00:00", "error_type": null, "error_message": null}
{"timestamp": "2026-08-13T00:18:36.765654+00:00", "level": "INFO", "logger": "URLMonitor", "message": "checking_url", "url": "https://httpbin.org/status/200", "attempt": 1}
{"timestamp": "2026-08-13T00:18:37.445363+00:00", "level": "INFO", "logger": "URLMonitor", "message": "url_checked", "url": "https://httpbin.org/status/200", "status": "ok", "status_code": 200, "response_time_ms": 679.65, "attempts": 1, "checked_at": "2026-08-13T00:18:37.445289+00:00", "error_type": null, "error_message": null}
Monitoramento concluido.
Total de URLs: 3
Sucesso: 3
Falhas: 0
PS C:\programacao\tarefa-04> |

```

TAREFA 5 -

Eu tentei executar a aplicação pelo Docker, mas tive um problema local com WSL no Docker Desktop. Para não deixar a tarefa sem validação, testei a aplicação diretamente com Python e confirmei que o monitor funciona. Mesmo assim, deixei os arquivos Dockerfile, docker-compose.yml e run.sh prontos para rodar em um ambiente com Docker.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\victo> cd "C:\programacao\tarefa-05"
PS C:\programacao\tarefa-05> $env:API_TOKEN="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"
PS C:\programacao\tarefa-05> python github_client.py --username octocat
Usuario: octocat
Repositorios listados: 5
Rate limit restante: 4998/5000
Arquivo output.json gerado.
PS C:\programacao\tarefa-05> |
```

Alternativa:

```
cd "C:\programacao\tarefa-04"
```

```
python monitor.py
```

TAREFA 6-

Um classificador de alertas. O sistema lê os alertas e extrai seus indicadores comparando com os dados da base local;

```
C:\programacao\tarefa-06>python processor.py
Processamento de alertas concluido.
Total de alertas: 3
Indicadores criticos: 3
Indicadores limpos: 4
Arquivo JSON: output.json
Arquivo CSV: output.csv
```

Nessa tarefa não foi encontrada dificuldades.

TAREFA 7-

A ideia foi criar um repositório local, fazer um commit inicial, criar uma branch de feature, implementar uma classe HealthChecker, fazer commit seguindo o padrão Conventional Commits e depois fazer merge para a branch main.

Também foi criado um CHANGELOG.md para registrar a funcionalidade.

Tive dificuldade para rodar o script Bash porque o WSL da minha máquina não estava configurado com uma distribuição Linux. Mesmo assim, o fluxo Git foi validado com comandos equivalentes no PowerShell.

TAREFA 8 -

Tarefa 8 é uma integração de todos os conhecimentos de tarefas anteriores;

Ele consulta uma api publica, depois processa esses dados e classifica por ip.

```

PS C:\Users\victo> cd "C:\programacao\tarefa-08-projeto-final"
PS C:\programacao\tarefa-08-projeto-final> python -m pip install -r requirements.txt
Requirement already satisfied: requests==2.32.3 in C:\Users\victo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packa
ges (from -r requirements.txt (line 1)) (2.32.3)
Requirement already satisfied: PyYAML==6.0.3 in C:\Users\victo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages
 (from -r requirements.txt (line 2)) (6.0.3)
Requirement already satisfied: charset-normalizer<4,>=2 in C:\Users\victo\AppData\Local\Python\pythoncore-3.14-64\Lib\si
te-packages (from requests==2.32.3->-r requirements.txt (line 1)) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in C:\Users\victo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages
 (from requests==2.32.3->-r requirements.txt (line 1)) (3.18)
Requirement already satisfied: urllib3<3,>=1.21.1 in C:\Users\victo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-pac
kages (from requests==2.32.3->-r requirements.txt (line 1)) (2.7.0)
Requirement already satisfied: certifi>=2017.4.17 in C:\Users\victo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-pac
kages (from requests==2.32.3->-r requirements.txt (line 1)) (2026.7.22)
PS C:\programacao\tarefa-08-projeto-final> python run-pipeline.py
{"timestamp": "2026-08-13T07:56:40.584276+00:00", "level": "INFO", "logger": "IPCollector", "message": "Consultando IP 8
.8.8.8, tentativa 1"}
{"timestamp": "2026-08-13T07:56:40.886950+00:00", "level": "INFO", "logger": "IPCollector", "message": "Consultando IP 1
.1.1.1, tentativa 1"}
{"timestamp": "2026-08-13T07:56:41.176771+00:00", "level": "INFO", "logger": "IPCollector", "message": "Consultando IP 1
85.220.101.45, tentativa 1"}
{"timestamp": "2026-08-13T07:56:41.475619+00:00", "level": "INFO", "logger": "IPCollector", "message": "Consultando IP 4
5.33.32.156, tentativa 1"}
Pipeline concluído. Resultado final: output\final_report.json

```